OPERATING SYSTEMS / HACKATHON

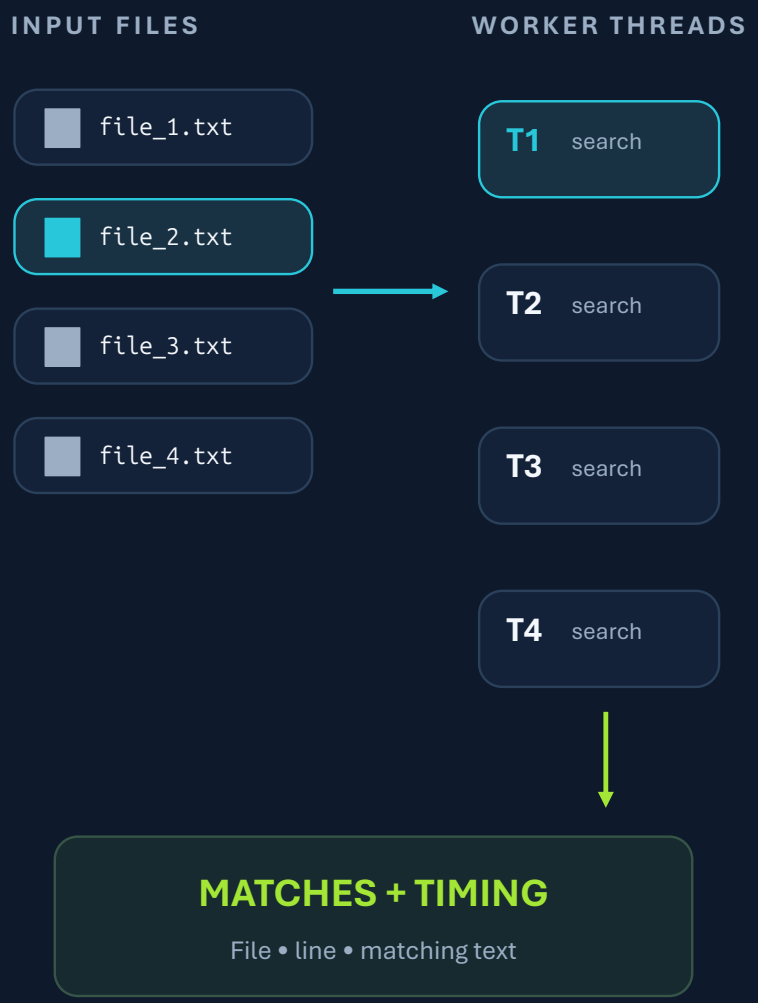
PARALLEL FILE SEARCH USING THREADS

A multithreaded text-file searcher built in C on Ubuntu Linux

Team Member 1: L. Sainath - 2520090148
Team Member 2: T. Shanmukha Sagar - 2520090050
Team Member 3: R. Nishanth - 2520090141

C LANGUAGE | POSIX THREADS | UBUNTU LINUX

Search concurrently. Synchronize safely. Measure honestly.



Why parallel file search?

Searching many files one by one can become slow as the dataset grows.

THE PROBLEM

A user needs to find a keyword across a directory of text files.

- Sequential search opens and scans each file in order.
- Large folders mean more file reads and more waiting.
- Manual search gives limited insight into execution performance.

PROJECT GOAL

Split independent file searches across worker threads.

- Find matching lines and show filename + line number.
- Keep shared task allocation thread-safe.
- Compare sequential and parallel elapsed time.

Core question: Can concurrent workers search the same dataset correctly—and when does it help?

What the prototype delivers

A compact, demonstrable implementation with observable OS concepts.

01

SEARCH

Case-insensitive keyword matching inside regular text files.

02

PARALLELIZE

Configurable number of POSIX worker threads.

03

REPORT

Matching file paths, line numbers, and matching text.

04

MEASURE

Sequential time, parallel time, counts, and speedup ratio.

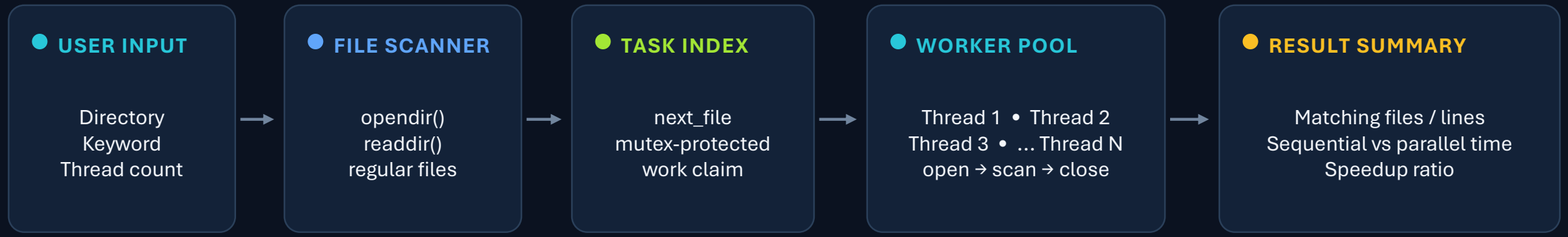
CURRENT SCOPE

One directory level • regular files • line-by-line scan • case-insensitive substring match

Recursive folder traversal and very long-line handling are future improvements.

From a folder to a combined result

The main thread prepares work; worker threads search separate files.



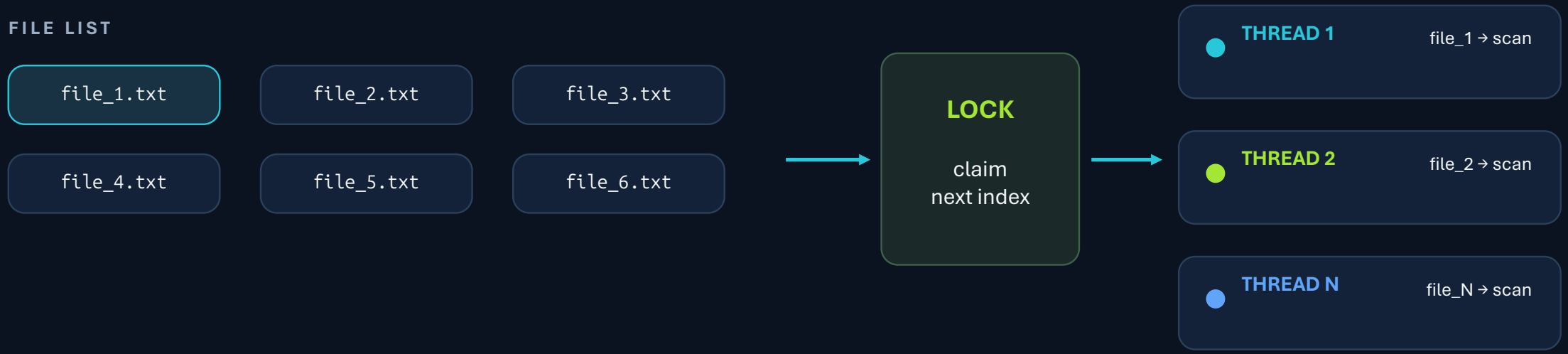
SYNCHRONIZATION POINTS

Task mutex — One worker claims each file index at a time.

Output mutex — Prevents console lines from interleaving unpredictably.

How worker threads divide the work

Each worker repeatedly claims the next file until the task list is exhausted.





Unique task indexes prevent two workers from intentionally claiming the same file.

Small functions, one clear pipeline

The implementation is built around collection, search, worker, timing, and reporting functions.

```
// Claim one file safely
pthread_mutex_lock(&task_mutex);
task = next_file++;
pthread_mutex_unlock(&task_mutex);

// Search assigned file
while (fgets(line, sizeof line, fp)) {
  if (strcasestr(line, keyword))
    print_match(file, line_number, line);
}
```

KEY FUNCTIONS

`collect_files()`	Builds the file list from a directory.
`search_file()`	Reads a file line by line and counts matches.
`worker()`	Claims jobs and updates thread-local statistics.
`parallel_search()`	Creates threads, joins them, and combines counters.
`sequential_search()`	Provides a baseline for comparison.

Key design choice: each thread accumulates local counts; the main thread combines them only after joins.

Built for the Ubuntu terminal

No external framework is required for the core search engine.

C	Core implementation	LANGUAGE
pthreads	Create / join worker threads	CONCURRENCY
Linux file APIs	Directory + file operations	FILE I/O
GCC	Compile and link the program	TOOLCHAIN
clock_gettime()	Monotonic elapsed-time timing	MEASUREMENT

```
$ gcc parallel_search.c -o parallel_search \
    -pthread -O2
$ ./parallel_search test_data error 4

directory   keyword   threads
```

OUTPUT INCLUDES

Matching paths • line numbers • total counts • elapsed time • speedup

Tip: quote a multiword phrase, e.g. "memory error".

Where synchronization matters

Protect shared state; keep independent file reads outside the task lock.

TASK MUTEX

Protects next_file

Only one worker claims the next index at a time. The lock is released before file I/O.

WHY IT MATTERS

Prevents duplicate task claims.

OUTPUT MUTEX

Protects console output

A worker locks only while printing a matching line, so output messages do not overlap.

WHY IT MATTERS

Keeps result lines readable.

THREAD JOIN

Waits for completion

The main thread joins each worker before combining that worker's final counters.

WHY IT MATTERS

Avoids reading incomplete totals.

Worker counters are local to each worker; their totals are combined after pthread_join().

Test correctness before claiming speed

Use the same file set and keyword for sequential and parallel runs.

TEST CASE	INPUT / ACTION	EXPECTED CHECK
Keyword found	Files contain “error”	Matching file + line are reported
Case-insensitive	Search “ERROR” or “error”	Counts should agree for mixed case
No match	Search a missing keyword	Zero matching lines and files
Thread counts	Run with 1, 2, 4, 8 workers	Same result totals as sequential
Empty directory	Directory has no regular files	Clear “no files” message
Invalid input	Bad path or non-positive threads	Validation/error message

Performance protocol: repeat the same run, record CPU / file count / total bytes, and avoid comparing different datasets.

What to report from a real run

Correctness can be checked with a known fixture; timing must be measured on your Ubuntu machine.

SAMPLE FIXTURE

Search keyword `error`

Expected matches in the 5 sample files

3 matching files

3 matching lines

PERFORMANCE COMPARISON

Sequential time	**[measured by program]**
Parallel time	**[measured by program]**

SPEEDUP FORMULA

Speedup = sequential elapsed time ÷ parallel elapsed time

Interpretation: more threads do not guarantee faster results—disk I/O, caching, thread overhead, and dataset size matter.

Where this approach can help

The project is a small, explainable pattern for parallelizing independent I/O tasks.

POTENTIAL APPLICATIONS

- **Log scanning** — Find error codes in application or system logs.
- **Codebase search** — Locate symbols or TODO markers in text source files.
- **Document repositories** — Search many text-based reports and notes.
- **Incident triage** — Quickly surface lines containing known warning terms.

LIMITATIONS → NEXT STEPS

CURRENT	Scans only the specified directory level.
CURRENT	Fixed-size line buffer can split very long lines.
IMPROVE	Add recursive traversal, extension filters, and file-size checks.
IMPROVE	Export benchmark results; handle cancellation and errors robustly.

Impact claim: faster search is possible for suitable workloads; the benchmark determines whether it happens for this dataset.

A 90-second demonstration flow

Show a repeatable test, then explain the concurrency controls.

1	**Prepare**	Create test_data with five small .txt files.
2	**Compile**	`gcc parallel_search.c -o parallel_search -pthread -O2`
3	**Run**	`./parallel_search test_data error 4`
4	**Verify**	Counts match the sequential baseline.
5	**Explain**	Task mutex • output mutex • pthread_join • speedup

TAKEAWAY

Threads divide work.

Mutexes protect shared state.
Joins make results ready.
Measurements guide the speed claim.

THANK YOU • QUESTIONS?